General Questions

- **Explain the difference** between a stored procedure and a stored function in MySQL, focusing on return values and how they are called.
- **Describe the advantages** of using stored routines (procedures and functions) in a database system, such as reducing network traffic and ensuring consistent updates.
- **Explain the difference** between a cursor and a trigger in MySQL, and the specific scenarios where you would use one over the other.

Stored Procedures

- **Practical Task**: Create a stored procedure named `GetEmployeeDetails` that accepts an `IN` parameter for `emp_id` and returns the employee's name, salary, and job title using `OUT` parameters or by returning a result set.
- **Practical Task**: Design a procedure that accepts two dates as input and returns the difference between them in a formatted string (e.g., "3 years 6 months").
- **Conceptual Question**: How can you handle errors or exceptions within a stored procedure in MySQL?

Functions

- **Practical Task**: Create a stored function named `CalculateAnnualSalary` that takes an employee's monthly salary and commission as input and returns their total annual salary (including commission).
- **Practical Task**: Write a function that returns the total number of active employees in a specific department, using the department ID as an input parameter.
- **Conceptual Question**: What keyword is used to create a stored function in MySQL, and how many values can a function return?

Triggers

- **Practical Task**: Create an `AFTER INSERT` trigger on an `orders` table. This trigger should automatically update the `stock_quantity` in the `products` table, reducing it by the quantity of the item just ordered.
- **Practical Task**: Write a `BEFORE INSERT` trigger to ensure that no new employee record is inserted if their `email` address is already present in the `employees` table.
- **Conceptual Question**: In which order are multiple triggers executed if they are defined for the same action on the same table?
- **Conceptual Question**: What type of operations (DML, DDL) can cause a trigger to execute automatically?

Cursors

- **Practical Task**: Write a stored procedure using a **cursor** to iterate through the `employees` table, identify employees whose salary is below a certain threshold (passed in as an argument), and insert their `emp_id` and a bonus amount into a separate `emp_bonus` table.
- **Conceptual Question**: Explain the lifecycle of a cursor in MySQL, including the necessary steps of declaration, opening, fetching data, and closing.
- **Conceptual Question**: When is it appropriate to use a cursor in MySQL?

Assessment: MySQL Stored Programs & Advanced Logic

1. Stored Procedures

- **Task 1: Basic Retrieval**
  Create a stored procedure named `GetEmployeesByDepartment` that accepts a department name as an `IN` parameter and retrieves all employee records belonging to that department.

- **Task 2: Data Manipulation**
  Write a procedure `UpdateEmployeeSalary` that takes an `emp_id` and a `percentage` (e.g., 10 for 10%). The procedure should increase that specific employee's salary by the given percentage.
- **Task 3: Output Parameters**
  Develop a procedure `CalculateTotalSalary` that takes a department ID as an `IN` parameter and uses an `OUT` parameter to return the total sum of salaries for that department.

2. Stored Functions

- **Task 4: Business Logic**
  Create a stored function `GetTaxAmount` that accepts a `salary` value and returns the calculated tax (e.g., 15% of the salary) as a `DECIMAL`.
- **Task 5: String Manipulation**
  Write a function `FormatEmployeeName` that accepts `first_name` and `last_name` and returns a single string formatted as "Lastname, Firstname" in all uppercase.

3. Triggers

- **Task 6: Audit Logging**
  Create an `AFTER UPDATE` trigger on an `Employees` table. Whenever a salary is changed, the trigger must insert the `emp_id`, `old_salary`, `new_salary`, and the current timestamp into a separate `SalaryAudit` table.
- **Task 7: Data Validation**
  Design a `BEFORE INSERT` trigger on a `Products` table. If the `stock_quantity` being inserted is less than zero, the trigger should automatically set it to zero to prevent invalid data.

4. Cursors

- **Task 8: Row-by-Row Processing**
  Create a stored procedure that uses a **cursor** to iterate through all rows in a

`Bonuses` table. For each row, if the employee's performance rating is 'Excellent', use the cursor to update their bonus to 20% of their base salary.

- **Task 9: Error Handling with Cursors**
  Enhance the previous procedure by implementing a `CONTINUE HANDLER` for the `NOT FOUND` condition to ensure the loop terminates correctly when all rows have been processed.

Theoretical Quick-Check

- **Q1:** What is the primary difference between a **Stored Procedure** and a **Stored Function** regarding return values?
- **Q2:** Can a **Trigger** be manually executed by a user, or does it only fire on specific events?
- **Q3:** Why are **Cursors** considered "read-only" and "non-scrollable" in MySQL?

Assessment: MySQL Stored Programs & Advanced Logic

1. Stored Procedures

- **Task 1: Basic Retrieval**
  Create a stored procedure named `GetEmployeesByDepartment` that accepts a department name as an `IN` parameter and retrieves all employee records belonging to that department.
- **Task 2: Data Manipulation**
  Write a procedure `UpdateEmployeeSalary` that takes an `emp_id` and a `percentage` (e.g., 10 for 10%). The procedure should increase that specific employee's salary by the given percentage.
- **Task 3: Output Parameters**
  Develop a procedure `CalculateTotalSalary` that takes a department ID as an `IN` parameter and uses an `OUT` parameter to return the total sum of salaries for that department.

2. Stored Functions

- **Task 4: Business Logic**
  Create a stored function `GetTaxAmount` that accepts a `salary` value and returns the calculated tax (e.g., 15% of the salary) as a `DECIMAL`.
- **Task 5: String Manipulation**
  Write a function `FormatEmployeeName` that accepts `first_name` and `last_name` and returns a single string formatted as "Lastname, Firstname" in all uppercase.

3. Triggers

- **Task 6: Audit Logging**
  Create an `AFTER UPDATE` trigger on an `Employees` table. Whenever a salary is changed, the trigger must insert the `emp_id`, `old_salary`, `new_salary`, and the current timestamp into a separate `SalaryAudit` table.
- **Task 7: Data Validation**
  Design a `BEFORE INSERT` trigger on a `Products` table. If the `stock_quantity` being inserted is less than zero, the trigger should automatically set it to zero to prevent invalid data.

4. Cursors

- **Task 8: Row-by-Row Processing**
  Create a stored procedure that uses a **cursor** to iterate through all rows in a `Bonuses` table. For each row, if the employee's performance rating is 'Excellent', use the cursor to update their bonus to 20% of their base salary.
- **Task 9: Error Handling with Cursors**
  Enhance the previous procedure by implementing a `CONTINUE HANDLER` for the `NOT FOUND` condition to ensure the loop terminates correctly when all rows have been processed.

Theoretical Quick-Check

- **Q1:** What is the primary difference between a **Stored Procedure** and a **Stored Function** regarding return values?
- **Q2:** Can a **Trigger** be manually executed by a user, or does it only fire on specific events?
- **Q3:** Why are **Cursors** considered "read-only" and "non-scrollable" in MySQL?